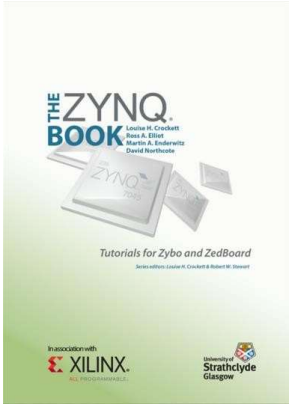
ECE3623 Embedded System Design Laboratory

Vivado AXI Interrupt



In this Laboratory you will utilize the embedded development of a Vivado Zynq Processor System (PS) with a hardware interrupt in a standalone OS. The tasks are described in detailed in Chapter 2 of the eText *The Zynq Book Tutorials* with the supporting files in the *The Zynq Book Tutorials Sources* both of which are posted on *Canvas* and the Lecture PowerPoints.

The Laboratory requires you to study and initially execute the Exercises 2A, 2B and 2C.

The Laboratory tasks are as follows:

1. Run the complete *interrupt_controller_tut_2B.c* Vivado/SDK project without modification to verify its performance in a standalone OS.

2. Modify the original *interrupt_controller_tut_2B.c* project to perform the following functions by configuring the *BTN_Intr_Handler* in SDK appropriately in a standalone OS:

   a) The Main Program executes a 4-bit down counter that repeatedly cycles. Use "*for (Delay = 0; Delay < LED_DELAY; Delay++);*" so that the LEDs blink at a visible rate.

   b) BTN0, upon an interrupt, stops the counter, turns OFF all the LEDs and then blinks all of the LEDs 5 times. After the interrupt is finished, the 4-bit counter continues from its last value. Use "*for (Delay = 0; Delay < LED_DELAY; Delay++);*" so that the LEDs blink at a visible rate.

See Shell Code Below.

Fall 2022

/*

```
 * interrupt_counter_tut_2B.c
 *
 *  Created on:         Unknown
 *      Author:  Ross Elliot
 *     Version:           1.1
 */

/************************************************************************************************

* VERSION HISTORY
************************************************************************************************
*        v1.1 - 01/05/2015
*                Updated for Zybo ~ DN
*
*        v1.0 - Unknown
*                First version created.
************************************************************************************************/

#include "xparameters.h"
#include "xgpio.h"
#include "xscugic.h"
#include "xil_exception.h"
#include "xil_printf.h"

// Parameter definitions
#define INTC_DEVICE_ID                  XPAR_PS7_SCUGIC_0_DEVICE_ID
#define BTNS_DEVICE_ID                  XPAR_AXI_GPIO_0_DEVICE_ID
#define LEDS_DEVICE_ID                  XPAR_AXI_GPIO_1_DEVICE_ID
#define INTC_GPIO_INTERRUPT_ID
XPAR_FABRIC_AXI_GPIO_0_IP2INTC_IRPT_INTR

#define BTN_INT                     XGPIO_IR_CH1_MASK

XGpio LEDInst, BTNInst;
XScuGic INTCInst;
static int led_data;
static int btn_value;

//_____________________________________
// PROTOTYPE FUNCTIONS
//_____________________________________
static void BTN_Intr_Handler(void *baseaddr_p);
static int InterruptSystemSetup(XScuGic *XScuGicInstancePtr);
static int IntcInitFunction(u16 DeviceId, XGpio *GpioInstancePtr);

//_____________________________________
```

```
// INTERRUPT HANDLER FUNCTIONS
// - called by the timer, button interrupt, performs
// - LED flashing
//_____________________________________-


void BTN_Intr_Handler(void *InstancePtr)
{
        // Disable GPIO interrupts
        XGpio_InterruptDisable(&BTNInst, BTN_INT);
        // Ignore additional button presses
        if ((XGpio_InterruptGetStatus(&BTNInst) & BTN_INT) !=
                        BTN_INT) {
                        return;
                }
        btn_value = XGpio_DiscreteRead(&BTNInst, 1);
        // Increment counter based on button value

        If(btn_value==0b0001){

        For(i=0; i<=5; i++){
        XGpio_DiscreteWrite(&LEDInst, 1, 0b0000);
        For (delay=0;delay<limit;delay++);
        XGpio_DiscreteWrite(&LEDInst, 1, 0b1111);
        For (delay=0;delay<limit;delay++);
        }
        }

    XGpio_DiscreteWrite(&LEDInst, 1, led_data);
    (void)XGpio_InterruptClear(&BTNInst, BTN_INT);
    // Enable GPIO interrupts
    XGpio_InterruptEnable(&BTNInst, BTN_INT);
}

//_____________________________________
// MAIN FUNCTION
//_____________________________________
int main (void)
{
  int status;
  //_____________________________________
  // INITIALIZE THE PERIPHERALS & SET DIRECTIONS OF GPIO
  //_____________________________________
  // Initialise LEDs
  status = XGpio_Initialize(&LEDInst, LEDS_DEVICE_ID);
  if(status != XST_SUCCESS) return XST_FAILURE;
```

```
  // Initialise Push Buttons
  status = XGpio_Initialize(&BTNInst, BTNS_DEVICE_ID);
  if(status != XST_SUCCESS) return XST_FAILURE;
  // Set LEDs direction to outputs
  XGpio_SetDataDirection(&LEDInst, 1, 0x00);
  // Set all buttons direction to inputs
  XGpio_SetDataDirection(&BTNInst, 1, 0xFF);

  // Initialize interrupt controller
  status = IntcInitFunction(INTC_DEVICE_ID, &BTNInst);
  if(status != XST_SUCCESS) return XST_FAILURE;

int led_output;
  while(1){
        led_output = led_output -1;
        XGpio_DiscreteWrite(&LEDInst, 1, led_output);
        For (delay=0;delay<limit;delay++);
        };

  return 0;
}

//_____________________________________
// INITIAL SETUP FUNCTIONS
//_____________________________________

int InterruptSystemSetup(XScuGic *XScuGicInstancePtr)
{
        // Enable interrupt
        XGpio_InterruptEnable(&BTNInst, BTN_INT);
        XGpio_InterruptGlobalEnable(&BTNInst);

        Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,

(Xil_ExceptionHandler)XScuGic_InterruptHandler,
                                                        XScuGicInstancePtr);
        Xil_ExceptionEnable();


        return XST_SUCCESS;

}

int IntcInitFunction(u16 DeviceId, XGpio *GpioInstancePtr)
{
        XScuGic_Config *IntcConfig;
```

```
	int status;

	// Interrupt controller initialisation
	IntcConfig = XScuGic_LookupConfig(DeviceId);
	status = XScuGic_CfgInitialize(&INTCInst, IntcConfig, IntcConfig-
>CpuBaseAddress);
	if(status != XST_SUCCESS) return XST_FAILURE;

	// Call to interrupt setup
	status =  InterruptSystemSetup(&INTCInst);
	if(status != XST_SUCCESS) return XST_FAILURE;

	// Connect GPIO interrupt to handler
	status = XScuGic_Connect(&INTCInst,
					INTC_GPIO_INTERRUPT_ID,

(Xil_ExceptionHandler)BTN_Intr_Handler,
					(void *)GpioInstancePtr);
	if(status != XST_SUCCESS) return XST_FAILURE;

	// Enable GPIO interrupts interrupt
	XGpio_InterruptEnable(GpioInstancePtr, 1);
	XGpio_InterruptGlobalEnable(GpioInstancePtr);

	// Enable GPIO and timer interrupts in the controller
	XScuGic_Enable(&INTCInst, INTC_GPIO_INTERRUPT_ID);

	return XST_SUCCESS;
}
```